

ETF 定价和对冲

一、定价

1.1 净值计算公式

以一天为一个时间周期(UTC+8 00:00:00至次日UTC+8 00:00:00),第k周期期末时刻记为 t_k , $k=0,1,2,\dots$, 其中 $t_0=0$ 。设单位基金初始净值为100USDT,即是, $S(0)=100$ USDT。

$S(0)$:基金初值;

$S(t)$:基金在t时刻的净值, $t \geq 0$;

$P(0)$:标的资产在初始时刻的价格;

$P(t)$:标的资产在t时刻的价格;

M:目标杠杆倍数,可取3、-3、5、-5。基金和标的资产在第k期的收益率分别为:

$$R_s(k) = \frac{S(t_k) - S(t_{k-1})}{S(t_{k-1})}, \quad R_p(k) = \frac{P(t_k) - P(t_{k-1})}{P(t_{k-1})}.$$

固定杠杆基金目标是使得其每一个时间周期的收益率为标的资产收益率的M倍,

即是 $R_s(k)=M \times R_p(k)$, $k=1,2, \dots$

基金在第k周期内的时刻v的净值通过以下公式计算:

$$S(v) = S(t_{k-1}) \times \left(1 + M \times \frac{P(v) - P(t_{k-1})}{P(t_{k-1})} \right), \quad \forall v \in [t_{k-1}, t_k).$$

固定杠杆产品本质上是一个主动管理产品,使其在每一个周期的回报率锚定对应标的资产回报率的M倍。

随着标的资产的价格变化,每周期期初的仓位必须做出调整,才能确保这一目标能实现。

仓位调整主要根据每周期期初的净值适当调整仓位,使得本周期的风险敞口锚定对应标的资产风险敞口的M倍。

如果当期起初的净值为 $S(t_k)$,期初设定的敞口则为 $M \times S(t_k)$ 的等量USDT仓位。

1.2 净值数据源

采用XT现货市场价格做为底层资产的价格数据源，现货市场定价采用中间价mid_price。

1.3 上新条件

1. 对冲市场币安有该标的合约产品，且对冲市场流动性需满足1%深度内流动性价值 ≥ 10 万U
2. XT交易所有该币对的现货市场

1.4 管理费

将日管理费，拆分为秒级收取，在净值中进行扣除

日管理费转化为秒级费率

首先，将日管理费转化为单位秒级费率

- 日管理费率：0.01%
- 转化为秒级费率

$$f_{\text{秒}} = \frac{f_{\text{日}}}{24 * 60 * 60}$$

净值的管理费扣除

$$NAV_{\text{decfee}} = NAV \times (1 - f_{\text{秒}})$$

1.5 净值计算代码

```
1  # -*- coding:utf-8 -*-
2
3  """
4  Author: AllenBrother
5  Date: 2024/11/28
6  Email: huangyongjie088@gmail.com
7  """
8
9  from strategies.etf.exchange.xt import Spot
10 from strategies.etf.utils.ds import Queue
11 from loguru import logger
```

```

12 import time
13
14
15 class NetValue:
16     def __init__(self, symbol, m_lever, init_net_value=1.0, daily_fee=0.001,
17 time_gap_second=10):
18         self.symbol = symbol
19         self.time_gap_fee = daily_fee / (24 * 60 * 60) * time_gap_second #
20 time_gap 管理费率
21         self.time_gap_second = time_gap_second
22         self.underlying_mid_price = 0
23         self.underlying_mid_price_queue = Queue(2)
24         self.net_value = {
25             "net_value": init_net_value,
26             "create_ts": int(time.time() * 1000),
27             "update_ts": 0
28         }
29         self.m_lever = m_lever
30
31     def update_mid_price(self):
32         depth = Spot(host="https://sapi.xt.com", access_key="",
33 secret_key="").get_depth(symbol=self.symbol, limit=5)
34         if depth:
35             bid_0_price = float(depth["bids"][0][0])
36             ask_0_price = float(depth["asks"][0][0])
37             mid_price = (bid_0_price + ask_0_price) / 2
38             logger.info(f"{self.symbol} mid_price:{mid_price}")
39             return mid_price
40         else:
41             logger.error(f"获取{self.symbol}市场深度失败")
42             return
43
44     def cal_net_value(self):
45         if len(self.underlying_mid_price_queue) == 2:
46             v = (self.underlying_mid_price_queue.get(1) -
47 self.underlying_mid_price_queue.get(0)) \
48             / self.underlying_mid_price_queue.get(0)
49             net_value = self.net_value["net_value"] * (1 + self.m_lever * v)
50
51             logger.info(
52                 f"{self.symbol} 底层资产价格变化({v}):
53 {self.underlying_mid_price_queue}")
54             return net_value
55         else:
56             logger.info(f"underlying_mid_price_queue:
57 {self.underlying_mid_price_queue} length != 2")
58             return None

```

```

53
54     def cal_fee(self, net_value):
55         net_value_dec_fee = net_value * (1 - self.time_gap_fee)
56         return net_value_dec_fee
57
58     def run(self):
59         while True:
60             mid_price = self.update_mid_price()
61             if not mid_price:
62                 continue # 没有获取到最新的数据, 直接返回
63             self.underlying_mid_price_queue.put(mid_price)
64             net_value = self.cal_net_value()
65             if not net_value:
66                 continue
67             # 调整管理费用
68             net_value_dec_fee = self.cal_fee(net_value)
69             self.net_value["net_value"] = net_value_dec_fee
70             self.net_value["update_ts"] = int(time.time() * 1000)
71             logger.info(f"{self.symbol} net_value:
{net_value}, net_value_dec_fee: {self.net_value}")
72             time.sleep(self.time_gap_second)
73
74
75 if __name__ == '__main__':
76     NetValue(symbol="btc_usdt", m_lever=3).run()

```

1.6 净值调整/仓位合并

1. 净值调整：因为净值的计算是按照10秒进行，属于连续累计过程。

假如，底层资产价格波动3%，日级 3L-ETF产品正常波动9%，但是秒级连续计算后，可能波动会偏离9%，所以此时应该对净值进行不定时调整。

这个调整的逻辑，暂时不确定，需要后续讨论。

2. 仓位合并：

杠杆 ETF 产品份额合并是指当产品净值过低，影响到交易便利性和价格精度时，为提升交易体验，会以合并份额的方式调整 ETF 产品净值和持仓量。该操作将影响用户 ETF 持仓数量和单位净值，但是不会影响用户的持仓总价值。

例如：

合并时刻，小明持有 **1,000 份 BTC3S**，每份净值为 **0.0025334**，总金额为 **2.5334 USDT**。合并后小明持有 1 份 BTC3S，每份净值为 2.5334，总金额保持不变，仍为 2.5334 USDT。

1.7 定价推送交易所接口

国 现货 ETF 接口文档

二、铺单

- 1. 根据定价作为铺单的中间价
- 2. 买卖价差可以调整
- 3. 其余可参考当前的铺单算法

三、对冲

3.1 对冲公式

做市仓位价值 + 对冲仓位价值 = 0

3.2 对冲流程

采用按照价格的事件变化动态对冲

